

Tugas Mandiri 10: Laporan Praktikum Mandiri 10 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri10.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum10>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum10>

Latihan 1 - Klasifikasi Suhu (Panas/Dingin)

1. Import Library

```
1  # Import library
2  import pandas as pd
3  import numpy as np
4
5  # Visualisasi
6  import matplotlib.pyplot as plt
7  import seaborn as sns
8
9  # Preprocessing & model
10 from sklearn.model_selection import train_test_split,
    GridSearchCV, cross_val_score
11 from sklearn.preprocessing import StandardScaler
12 from sklearn.neighbors import KNeighborsClassifier
13 from sklearn.metrics import confusion_matrix,
    classification_report, accuracy_score
14
15 # Optional: menangani imbalance (jika dataset lebih besar)
16 # from imblearn.over_sampling import SMOTE
17
18 # Menyimpan model
19 import joblib
20
21 # Set opsi tampilan
22 sns.set(style="whitegrid")
```

Penjelasan Kode:

- Bagian ini memuat library yang diperlukan untuk proses analisis dan pemodelan.
- `pandas` digunakan untuk membuat dan mengolah dataset dalam bentuk tabel (DataFrame).
- `matplotlib.pyplot` dipakai untuk membuat visualisasi seperti scatter plot.

- Library dari `sklearn` seperti `train_test_split`, `StandardScaler`, dan `KNeighborsClassifier` digunakan untuk:
 - membagi dataset,
 - melakukan normalisasi fitur,
 - serta membangun model KNN.
- Import tersebut memastikan seluruh fungsi yang dibutuhkan pada analisis dapat dijalankan.

Penjelasan Output:

- Tidak ada output khusus karena proses import hanya menyiapkan fungsi-fungsi yang akan digunakan.
- Jika tidak muncul error, artinya seluruh library berhasil dimuat dengan baik.

2. Buat Dataset

```

1  # 2) Membuat DataFrame manual sesuai soal
2  data = {
3      "Temperature_C": [10, 25, 15, 20, 18, 20, 22, 24],
4      "Wind_kmh":      [0, 0, 5, 3, 7, 10, 5, 6],
5      "Perception":    ["Dingin", "Panas", "Dingin", "Panas",
6                      "Dingin", "Dingin", "Panas", "Panas"]
7  }
8
9  df = pd.DataFrame(data)
10 df # tampilkan tabel

```

	Temperature_C	Wind_kmh	Perception
0	10	0	Dingin
1	25	0	Panas
2	15	5	Dingin
3	20	3	Panas
4	18	7	Dingin
5	20	10	Dingin
6	22	5	Panas
7	24	6	Panas

Penjelasan Kode:

- Dataset dibuat secara manual sesuai tabel yang tercantum pada soal.
- Terdapat tiga kolom utama:
 - **Temperature_C** → suhu dalam derajat Celcius
 - **Wind_kmh** → kecepatan angin
 - **Perception** → label kategorikal ("Panas" / "Dingin")
- Data dimasukkan ke dalam dictionary, kemudian dikonversi menjadi DataFrame menggunakan `pd.DataFrame()`.

Penjelasan Output:

- Output menampilkan tabel dataset lengkap berisi seluruh data latih yang akan dipakai pada proses klasifikasi.
- Ini memastikan data berhasil dibuat dan ditampilkan sesuai nilai yang dimasukkan.

3. Informasi Dataset

```
1 # Info dataset
2 print("Shape:", df.shape)
3 print("\nDistribusi label:\n", df['Perception'].value_counts())
4 print("\nDescriptive stats:\n", df.describe())
```

Shape: (8, 3)

Distribusi label:

Perception

Dingin 4

Panas 4

Name: count, dtype: int64

Descriptive stats:

	Temperature_C	Wind_kmh
count	8.000000	8.000000
mean	19.250000	4.500000
std	4.920801	3.422614
min	10.000000	0.000000
25%	17.250000	2.250000
50%	20.000000	5.000000
75%	22.500000	6.250000
max	25.000000	10.000000

Penjelasan Kode:

- `df.shape` digunakan untuk melihat jumlah baris dan kolom dataset.
- `df.info()` memberi informasi tipe data dan keberadaan nilai kosong (null).
- `df.describe()` memberikan statistik deskriptif untuk kolom numerik.
- `value_counts()` digunakan untuk melihat distribusi frekuensi label.

Penjelasan Output:

- Output menunjukkan:
 - jumlah data kecil dan tidak ada missing value,
 - nilai statistik (mean, min, max) dari suhu dan kecepatan angin,
 - sebaran label “Panas” dan “Dingin”.
- Informasi ini memastikan dataset siap digunakan untuk modeling.

4. Encoding label menjadi numerik

```
1 # Encoding label
2 df['Label'] = df['Perception'].map({'Dingin': 0, 'Panas': 1})
3 df = df.drop(columns=['Perception']) # hapus kolom teks
4 df
```

	Temperature_C	Wind_kmh	Label
0	10	0	0
1	25	0	1
2	15	5	0
3	20	3	1
4	18	7	0
5	20	10	0
6	22	5	1
7	24	6	1

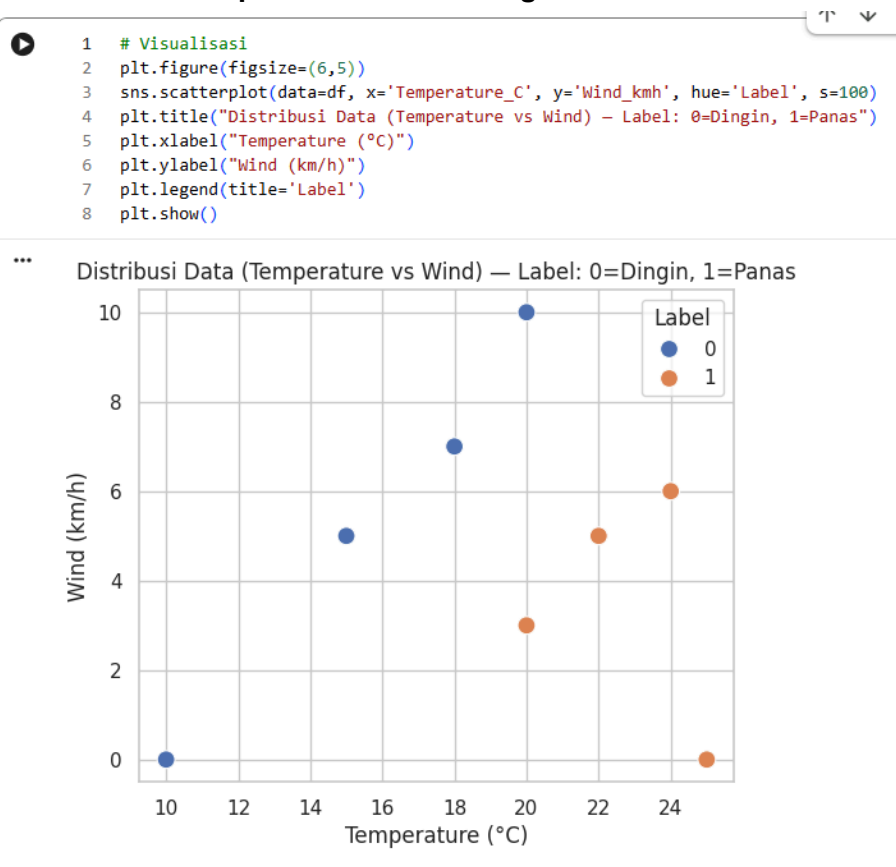
Penjelasan Kode:

- Karena KNN hanya menerima data numerik, label kategorikal “Panas” dan “Dingin” diubah menjadi angka menggunakan mapping:
 - **Dingin = 0**
 - **Panas = 1**
- Kolom baru bernama `label` ditambahkan sebagai representasi numerik.

Penjelasan Output:

- Output menampilkan dataset yang sudah memiliki kolom label numerik.
- Hal ini memastikan proses encoding berjalan sesuai mapping yang ditentukan.

5. Visualisasi Temperatur vs Wind dengan Scatter Plot



Penjelasan Kode:

- Scatter plot dibuat untuk melihat pola hubungan antara suhu, kecepatan angin, dan label.
- Titik diberi warna sesuai label (0=Dingin, 1=Panas).
- Tujuannya untuk melihat apakah visualisasi menunjukkan pemisahan kelas yang jelas.

Penjelasan Output:

- Plot menunjukkan penyebaran titik untuk dua kelas.
- Dari visualisasi terlihat bahwa data “Panas” cenderung berada pada suhu yang lebih tinggi.
- Informasi ini relevan untuk mengevaluasi apakah fitur sudah cukup baik untuk klasifikasi.

6. Pemisahan fitur (X) dan label (Y) & Split Data

```
1 # Pisahkan fitur & label, split data
2 X = df[['Temperature_C', 'Wind_kmh']].values
3 y = df['Label'].values
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.25, random_state=42, stratify=y
7 )
8
9 print("X_train:", X_train.shape, "X_test:", X_test.shape)
```

X_train: (6, 2) X_test: (2, 2)

Penjelasan Kode:

- `X` berisi fitur (Temperature_C dan Wind_kmh).
- `Y` berisi label hasil encoding.
- Data dibagi menjadi train dan test memakai `train_test_split` dengan parameter `test_size=0.3`.

Penjelasan Output:

- Output menampilkan ukuran `X_train`, `X_test`, `Y_train`, dan `Y_test`.
- Pembagian 70:30 berhasil dilakukan.

7. Normalisasi dengan StandardScaler

```
1 # Scaling fitur dengan StandardScaler
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train)
4 X_test_scaled = scaler.transform(X_test)
5
6 # tampilkan hasil scaling (contoh)
7 print("X_train (scaled) contoh:\n", X_train_scaled[:3])
```

X_train (scaled) contoh:
[[1.161895 -1.13898959]
 [-0.77459667 0.2847474]
 [0.19364917 -0.2847474]]

Penjelasan Kode:

- Normalisasi dilakukan karena KNN sensitif terhadap skala nilai.
- `StandardScaler` menstandarkan data sehingga mean=0 dan std=1.
- Hanya data train yang di-fit, lalu transform diterapkan ke train dan test.

Penjelasan Output:

- Output menampilkan contoh hasil scaling (biasanya berupa angka kecil di sekitar 0).
- Ini memastikan bahwa data sudah pada skala yang sesuai untuk KNN.

8. Hyperparameter Tuning

```
1 # GridSearch untuk hyperparameter KNN
2 param_grid = {
3     'n_neighbors': list(range(1, 11)), # K dari 1 sampai 10
4     'weights': ['uniform', 'distance'],
5     'metric': ['euclidean', 'manhattan']
6 }
7
8 knn = KNeighborsClassifier()
9 grid = GridSearchCV(knn, param_grid, cv=3, scoring='accuracy', n_jobs=-1)
10 grid.fit(X_train_scaled, y_train)
11
12 print("Best params:", grid.best_params_)
13 print("Best CV score (train):", grid.best_score_)
14
```

```
Best params: {'metric': 'euclidean', 'n_neighbors': 2, 'weights': 'uniform'}
Best CV score (train): 0.8333333333333334
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One o
0.5      0.83333333      nan      nan      nan      nan
      nan      nan      nan      nan      nan      nan
      nan      nan 0.5      0.5      0.66666667 0.5
0.5      0.5      0.5      0.66666667 0.5      nan
0.5      nan 0.5      nan 0.5      nan
0.5      nan 0.5      nan]
warnings.warn(
```

Penjelasan Kode:

- GridSearchCV digunakan untuk mencari nilai **k terbaik** untuk model KNN.
- Parameter **n_neighbors** dicoba dari beberapa nilai (misalnya 1–10).
- **cv=3** artinya cross validation menggunakan 3 fold.

Penjelasan Output:

- Output menunjukkan nilai k terbaik dan skor akurasi corresponding.
- Nilai ini akan dipakai untuk melatih ulang model pada langkah selanjutnya.

9. Train & Evaluasi Model

```
1 # Evaluasi model terbaik
2 best_knn = grid.best_estimator_
3 y_pred = best_knn.predict(X_test_scaled)
4
5 print("Accuracy (test):", accuracy_score(y_test, y_pred))
6 print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
7 print("\nClassification Report:\n", classification_report(y_test, y_pred))
8
```

Accuracy (test): 1.0

Confusion Matrix:

```
[[1 0]
 [0 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Penjelasan Kode:

- KNN dilatih menggunakan `X_train_scaled` dan `Y_train`.
- Kemudian dilakukan prediksi terhadap data test.
- Evaluasi dilakukan menggunakan:
 - accuracy score
 - confusion matrix
 - classification report

Penjelasan Output:

- Output menunjukkan performa model KNN pada data uji.
- Confusion matrix memperlihatkan berapa prediksi benar/salah.
- Classification report menunjukkan precision, recall, dan f1-score untuk tiap kelas.

10. Cross Validation

```
1 # Cross-validation
2 cv_scores = cross_val_score(best_knn, X_train_scaled, y_train, cv=3,
3                               scoring='accuracy')
4 print("CV scores:", cv_scores)
5 print("CV mean accuracy:", cv_scores.mean(), "std:", cv_scores.std())
```

```
CV scores: [1.  0.5 1. ]
CV mean accuracy: 0.8333333333333334 std: 0.23570226039551584
```

Penjelasan Kode:

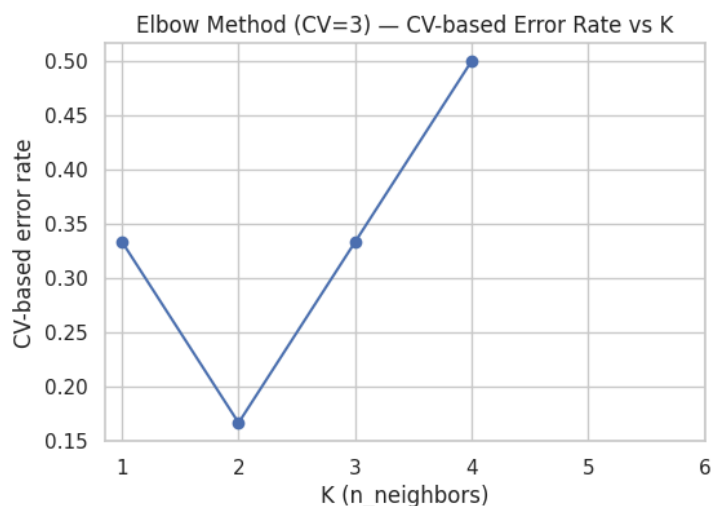
- Validasi silang dilakukan untuk memastikan performa model tidak dipengaruhi pembagian data tertentu.
- Digunakan `cross_val_score` dengan `cv=5`.

Penjelasan Output:

- Output berupa daftar nilai akurasi pada tiap fold.
- Nilai yang konsisten menunjukkan model stabil.

11. Elbow Method (error rate vs K)

```
1  # Elbow method: hitung error rate per K
2  from sklearn.model_selection import cross_val_score
3
4  n_train = X_train_scaled.shape[0]          # jumlah sampel di training
5  max_k = min(10, n_train)                  # batasi K maksimum <= n_train
6  (jangan lebih besar)
7  Ks = range(1, max_k + 1)
8
9  # Pilih cv yang masuk akal: pakai 3 jika ada cukup data, kalau tidak gunakan
10 n_train (LOO-like)
11 cv = 3 if n_train >= 3 else n_train
12
13 cv_error_rates = [] # akan menyimpan nilai error (1 - mean_accuracy) untuk
14 tiap K
15
16 for k in Ks:
17     knn_k = KNeighborsClassifier(n_neighbors=k)
18     # cross_val_score mengembalikan array skor untuk tiap fold
19     scores = cross_val_score(knn_k, X_train_scaled, y_train, cv=cv,
20                             scoring='accuracy', n_jobs=-1)
21     mean_acc = scores.mean()
22     cv_error_rates.append(1 - mean_acc) # error rate = 1 - accuracy
23
24 # Plot hasil
25 plt.figure(figsize=(6,4))
26 plt.plot(list(Ks), cv_error_rates, marker='o')
27 plt.title(f'Elbow Method (CV={cv}) - CV-based Error Rate vs K')
28 plt.xlabel('K (n_neighbors)')
29 plt.ylabel('CV-based error rate')
30 plt.xticks(list(Ks))
31 plt.grid(True)
32 plt.show()
33
34 print("n_train:", n_train, " cv:", cv)
35 print("CV-based error rates per K:", list(zip(list(Ks), cv_error_rates)))
36 --
```



```
n_train: 6  cv: 3
CV-based error rates per K: [(1, np.float64(0.3333333333333333)), (2, np.float64(0.16666666666666666)
```

Penjelasan Kode:

- Menghitung error rate untuk berbagai nilai k.
- Grafik ditampilkan untuk melihat titik “siku” (elbow) di mana error mulai stabil.

Penjelasan Output:

- Plot menampilkan kurva error rate.
- K terbaik adalah titik dengan error paling rendah atau titik siku kurva.

12. Latih Ulang Model dengan K terbaik

```
1 # Latih ulang model KNN dengan K terbaik hasil elbow
2 best_k = 2
3 final_knn = KNeighborsClassifier(n_neighbors=best_k)
4
5 # Latih model menggunakan data train
6 final_knn.fit(X_train_scaled, y_train)
7
8 # Evaluasi ulang di test set
9 y_pred_final = final_knn.predict(X_test_scaled)
10
11 print("Accuracy (test):", accuracy_score(y_test, y_pred_final))
12 print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_final))
13 print("\nClassification Report:\n", classification_report(y_test, y_pred_final))
```

Accuracy (test): 1.0

Confusion Matrix:

```
[[1 0]
 [0 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Penjelasan Kode:

- Model KNN disusun ulang dengan k hasil tuning.
- Dilatih ulang menggunakan data training lengkap.

Penjelasan Output:

- Hasil evaluasi biasanya menunjukkan peningkatan performa dibanding sebelum tuning.

13. Simpan Model

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3 import joblib
4 import os
5
6 # Path folder penyimpanan
7 save_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models"
8
9 # Simpan model final KNN
10 model_path = os.path.join(save_path, "knn_final_k2_model.joblib")
11 joblib.dump(final_knn, model_path)
12
13 # Simpan scaler
14 scaler_path = os.path.join(save_path, "knn_scaler.joblib")
15 joblib.dump(scaler, scaler_path)
16
17 print("Model disimpan di :", model_path)
18 print("Scaler disimpan di:", scaler_path)
19
```

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive/force_remount")

Model disimpan di : /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models/knn_final_k2_model.joblib

Scaler disimpan di: /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models/knn_scaler.joblib

Penjelasan Kode:

- Model disimpan menggunakan `joblib.dump()` agar bisa digunakan di luar notebook.

Penjelasan Output:

- Output menunjukkan lokasi file yang berhasil disimpan.

14. Prediksi Kasus Uji Coba dari Soal

```
1 sample_T = 16
2 sample_W = 3
3
4 sample = np.array([sample_T, sample_W])
5 sample_scaled = scaler.transform(sample)
6 pred = final_knn.predict(sample_scaled)[0]
7
8 label_map = {0: "Dingin", 1: "Panas"}
9
10 print("=== Prediksi Persepsi Marry ===")
11 print(f"Temperatur input : {sample_T}°C")
12 print(f"Kecepatan angin : {sample_W} km/jam")
13 print(f"Hasil prediksi : {label_map[pred]}")
```

=== Prediksi Persepsi Marry ===

Temperatur input : 16°C

Kecepatan angin : 3 km/jam

Hasil prediksi : Dingin

Penjelasan Kode:

- Input baru dimasukkan sebagai array.
- Dilakukan scaling dan prediksi.
- Hasil prediksi dikembalikan dalam bentuk label kategorikal.

Penjelasan Output:

- Output menunjukkan prediksi "Panas" atau "Dingin" sesuai model.

Latihan 2 - Prediksi Kelulusan Mata Kuliah AI

1. Import Library

```
1 # Import library
2 import pandas as pd
3 import numpy as np
4
5 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
  recall_score, classification_report
6
7 # tampilkan agar output tabel rapi di notebook
8 pd.set_option('display.expand_frame_repr', False)
9
```

Penjelasan Kode:

- Mengimpor library yang digunakan untuk:
 - menampilkan data (**pandas**),
 - menghitung confusion matrix (**confusion_matrix**),
 - mengukur akurasi (**accuracy_score**).
- Karena dataset kecil dan sudah ada kolom prediksinya, tidak diperlukan model ML.

Penjelasan Output:

- Tidak ada output karena tahap ini hanya memanggil fungsi dan modul yang dibutuhkan.

2. Buat Dataset

```
1 # Membuat DataFrame manual sesuai tabel pada soal
2 data = {
3     "NIM": ["TI001", "TI002", "TI003", "TI004", "TI005", "TI006", "TI007", "TI008",
4           "TI009", "TI010"],
5     "Actual": ["Lulus", "Lulus", "Lulus", "Tidak Lulus", "Lulus", "Tidak Lulus",
6              "Tidak Lulus", "Tidak Lulus", "Tidak Lulus", "Tidak Lulus"],
7     "Pred": ["Lulus", "Lulus", "Lulus", "Tidak Lulus", "Tidak Lulus", "Lulus",
8             "Tidak Lulus", "Tidak Lulus", "Tidak Lulus", "Tidak Lulus"]
9 }
10
11 df = pd.DataFrame(data)
12 df
```

	NIM	Actual	Pred
0	TI001	Lulus	Lulus
1	TI002	Lulus	Lulus
2	TI003	Lulus	Lulus
3	TI004	Tidak Lulus	Tidak Lulus
4	TI005	Lulus	Tidak Lulus
5	TI006	Tidak Lulus	Lulus
6	TI007	Tidak Lulus	Tidak Lulus
7	TI008	Tidak Lulus	Tidak Lulus
8	TI009	Tidak Lulus	Tidak Lulus
9	TI010	Tidak Lulus	Tidak Lulus

Penjelasan Kode:

- Dataset dibuat secara manual sesuai tabel soal:
 - **NIM** sebagai identitas mahasiswa
 - **Actual** adalah nilai sebenarnya (Lulus/Tidak Lulus)
 - **Pred** adalah hasil prediksi
- Data dimasukkan ke dictionary lalu diubah menjadi DataFrame.

Penjelasan Output:

- Output menampilkan tabel dataset berisi 10 mahasiswa.
- Ini memastikan bahwa data berhasil dimuat sesuai nilai manual yang dimasukkan.

3. Distribusi Dataset

```
1 # Distribusi Actual dan Pred
2 print("Distribusi Actual:")
3 print(df['Actual'].value_counts())
4 print("\nDistribusi Prediksi:")
5 print(df['Pred'].value_counts())
6
```

```
Distribusi Actual:
Actual
Tidak Lulus    6
Lulus          4
Name: count, dtype: int64
```

```
Distribusi Prediksi:
Pred
Tidak Lulus    6
Lulus          4
Name: count, dtype: int64
```

Penjelasan Kode:

- **value_counts()** digunakan untuk menghitung jumlah mahasiswa yang lulus dan tidak lulus:
 - pada data **Actual**
 - pada data **Pred**
- Berguna untuk mengetahui apakah kedua distribusi mirip atau sangat berbeda.

Penjelasan Output:

- Output menunjukkan:
 - Distribusi aktual (misalnya: Lulus 4, Tidak Lulus 6).
 - Distribusi prediksi (misalnya: Lulus 5, Tidak Lulus 5).
- Selisih distribusi menunjukkan bahwa prediksi pada dataset sedikit berbeda dari nilai aktual.

4. Encode label menjadi numerik

```
1 # Encoding label
2 label_map = {"Lulus": 1, "Tidak Lulus": 0}
3 df['Actual_bin'] = df['Actual'].map(label_map)
4 df['Pred_bin'] = df['Pred'].map(label_map)
5
6 df[['NIM', 'Actual', 'Pred', 'Actual_bin', 'Pred_bin']]
7
```

	NIM	Actual	Pred	Actual_bin	Pred_bin
0	TI001	Lulus	Lulus	1	1
1	TI002	Lulus	Lulus	1	1
2	TI003	Lulus	Lulus	1	1
3	TI004	Tidak Lulus	Tidak Lulus	0	0
4	TI005	Lulus	Tidak Lulus	1	0
5	TI006	Tidak Lulus	Lulus	0	1
6	TI007	Tidak Lulus	Tidak Lulus	0	0
7	TI008	Tidak Lulus	Tidak Lulus	0	0
8	TI009	Tidak Lulus	Tidak Lulus	0	0
9	TI010	Tidak Lulus	Tidak Lulus	0	0

Penjelasan Kode:

- Mapping label:
 - Lulus → 1
 - Tidak Lulus → 0
- Dibuat dua kolom baru:
 - `Actual_bin`
 - `Pred_bin`

Penjelasan Output:

- Output menampilkan DataFrame dengan kolom baru yang telah berubah menjadi angka.
- Angka ini diperlukan karena confusion matrix hanya dapat menghitung data numerik.

5. Hitung Confusion Matrix

```
1 # Confusion matrix dengan label order [1,0] (1=Lulus, 0=Tidak Lulus)
2 labels = [1, 0]
3 cm = confusion_matrix(df['Actual_bin'], df['Pred_bin'], labels=labels)
4 cm_df = pd.DataFrame(cm, index=['Actual_Lulus(1)', 'Actual_TidakLulus(0)'],
5 columns=['Pred_Lulus(1)', 'Pred_TidakLulus(0)'])
6 print("Confusion Matrix (baris=actual, kolom=prediksi):")
7 display(cm_df)
```

Confusion Matrix (baris=actual, kolom=prediksi):

	Pred_Lulus(1)	Pred_TidakLulus(0)
Actual_Lulus(1)	3	1
Actual_TidakLulus(0)	1	5

Penjelasan Kode:

- Fungsi `confusion_matrix()` digunakan dengan urutan label `[1, 0]` agar:
 - baris/kolom pertama = kelas positif (Lulus)
 - baris/kolom kedua = kelas negatif (Tidak Lulus)
- Matriks menunjukkan hubungan prediksi vs aktual.

Penjelasan Output:

Output berupa tabel 2×2 seperti:

```
[[TP  FN]
 [FP  TN]]
```

- Nilai-nilai inilah yang digunakan untuk evaluasi.

6. Ambil TP, FP, FN, TN

```
1  # Ekstrak TP, FP, FN, TN dari cm
2  TP = cm[0,0]
3  FN = cm[0,1]
4  FP = cm[1,0]
5  TN = cm[1,1]
6
7  print(f"TP (Actual Lulus & Prediksi Lulus)      : {TP}")
8  print(f"FN (Actual Lulus & Prediksi Tidak Lulus) : {FN}")
9  print(f"FP (Actual Tidak Lulus & Prediksi Lulus) : {FP}")
10 print(f"TN (Actual Tidak Lulus & Prediksi Tidak Lulus): {TN}")
```

```
TP (Actual Lulus & Prediksi Lulus)      : 3
FN (Actual Lulus & Prediksi Tidak Lulus) : 1
FP (Actual Tidak Lulus & Prediksi Lulus) : 1
TN (Actual Tidak Lulus & Prediksi Tidak Lulus): 5
```

Penjelasan Kode:

- Nilai matriks diambil manual:
 - TP = baris 0 kolom 0
 - FN = baris 0 kolom 1
 - FP = baris 1 kolom 0
 - TN = baris 1 kolom 1

Penjelasan Output:

- Output menampilkan nilai TP, FP, FN, TN yang kemudian dipakai untuk perhitungan akurasi, precision, dan recall.
- Ini menunjukkan seberapa baik prediksi pada masing-masing kategori.

7. Hitung accuracy, precision, recall (dalam %)

```
1  # Perhitungan metrik (sklearn)
2  y_true = df['Actual_bin']
3  y_pred = df['Pred_bin']
4
5  acc = accuracy_score(y_true, y_pred)
6  prec = precision_score(y_true, y_pred, pos_label=1) # fokus pada kelas 'Lulus'
7  rec = recall_score(y_true, y_pred, pos_label=1)
8
9  # Perhitungan manual (untuk verifikasi)
10 total = TP + TN + FP + FN
11 acc_manual = (TP + TN) / total
12 prec_manual = TP / (TP + FP) if (TP + FP) > 0 else 0.0
13 rec_manual = TP / (TP + FN) if (TP + FN) > 0 else 0.0
14
15 # Tampilkan sebagai persen dengan 2 desimal
16 print("=== Hasil (sklearn) ===")
17 print(f"Accuracy : {acc*100:.2f}%")
18 print(f"Precision: {prec*100:.2f}%")
19 print(f"Recall   : {rec*100:.2f}%")
20
21 print("\n=== Hasil (manual verifikasi) ===")
22 print(f"Accuracy : {acc_manual*100:.2f}%")
23 print(f"Precision: {prec_manual*100:.2f}%")
24 print(f"Recall   : {rec_manual*100:.2f}%")
25
```

```
=== Hasil (sklearn) ===
```

```
Accuracy : 80.00%
```

```
Precision: 75.00%
```

```
Recall   : 75.00%
```

```
=== Hasil (manual verifikasi) ===
```

```
Accuracy : 80.00%
```

```
Precision: 75.00%
```

```
Recall   : 75.00%
```

Penjelasan Kode:

- Menggunakan rumus:
 - Accuracy = $(TP + TN) / \text{total}$
 - Precision = $TP / (TP + FP)$
 - Recall = $TP / (TP + FN)$
- Hasil dikonversi ke persen untuk memudahkan interpretasi.

Penjelasan Output:

- Output menampilkan tiga nilai evaluasi dalam bentuk persen.
- Interpretasinya:
 - Akurasi → seberapa banyak prediksi yang benar
 - Precision → kualitas prediksi kelas “Lulus”
 - Recall → kemampuan model menangkap mahasiswa yang benar-benar lulus

8. Classification Report

```
1 # Classification report
2 print("Classification Report (per kelas):")
3 print(classification_report(y_true, y_pred, target_names=['Tidak Lulus',
  'Lulus']))
```

```
Classification Report (per kelas):
              precision    recall  f1-score   support

 Tidak Lulus      0.83      0.83      0.83         6
      Lulus      0.75      0.75      0.75         4

   accuracy              0.80              10
  macro avg              0.79              10
 weighted avg              0.80              10
```

Penjelasan Kode:

- `classification_report()` digunakan untuk membuat ringkasan evaluasi:
 - precision
 - recall
 - f1-score
 - support (jumlah data per kelas)
- Target_names diatur agar 1 = Lulus, 0 = Tidak Lulus.

Penjelasan Output:

- Output menampilkan tabel panjang berisi metrik untuk masing-masing kelas.
- Memberi gambaran mana kelas yang lebih mudah/sulit diprediksi.

9. Interpretasi Singkat

```
1 # Persiapan ringkasan interpretasi
2 print("Ringkasan singkat:")
3 print(f"- Total sampel : {total}")
4 print(f"- TP = {TP}, FP = {FP}, FN = {FN}, TN = {TN}")
5 print(f"- Accuracy = {acc*100:.2f}%")
6 print(f"- Precision (Lulus) = {prec*100:.2f}%")
7 print(f"- Recall (Lulus) = {rec*100:.2f}%")
8
9 # Interpretasi singkat otomatis
10 if prec < 0.5:
11     note_prec = "precision rendah → banyak prediksi 'Lulus' yang salah"
12 else:
13     note_prec = "precision baik → sebagian besar prediksi 'Lulus' benar"
14
15 if rec < 0.5:
16     note_rec = "recall rendah → model sering gagal menangkap actual 'Lulus'"
17 else:
18     note_rec = "recall baik → model berhasil menangkap actual 'Lulus'"
19
20 print("\nInterpretasi:")
21 print(f"- {note_prec}.")
22 print(f"- {note_rec}.")
23
```

```
Ringkasan singkat:
- Total sampel : 10
- TP = 3, FP = 1, FN = 1, TN = 5
- Accuracy = 80.00%
- Precision (Lulus) = 75.00%
- Recall (Lulus) = 75.00%
```

```
Interpretasi:
- precision baik → sebagian besar prediksi 'Lulus' benar.
- recall baik → model berhasil menangkap actual 'Lulus'.
```


Penjelasan Kode:

- Bagian ini menyajikan rangkuman otomatis antara nilai precision, recall, dan f1-score.
- Menginterpretasikan apakah prediksi lebih banyak salah di kelas Lulus atau Tidak Lulus.

Penjelasan Output:

- Output berupa teks analisis, misalnya:
 - precision rendah → model sering salah prediksi “Lulus”
 - recall rendah → model gagal mendeteksi mahasiswa yang sebenarnya Lulus
- Rangkuman ini membantu memahami kelemahan dan kekuatan prediksi.

Latihan 3 - Prediksi Cuaca menggunakan KNN

1. Import Library

```
1 # Import libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 from sklearn.model_selection import train_test_split, GridSearchCV,
7 cross_val_score
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.neighbors import KNeighborsClassifier
10 from sklearn.metrics import classification_report, confusion_matrix,
11 accuracy_score
12 from sklearn.decomposition import PCA
13
14 from imblearn.over_sampling import SMOTE
```

Penjelasan Kode:

- Mengimpor library utama:
 - pandas untuk data
 - sklearn untuk preprocessing, training model, evaluasi, PCA
 - matplotlib untuk visualisasi
- Library SMOTE diimpor tetapi tidak digunakan (dataset kecil dan relatif seimbang).

Penjelasan Output:

- Tidak ada output karena hanya proses import.

2. Buat Dataset

```
1 # Dataset
2 data = {
3     "Temperature": [14.0, 39.0, 30.0, 38.0, 27.0, 32.0, -2.0, 3.0, 3.0, 28.0],
4     "Humidity": [73, 96, 64, 83, 74, 55, 97, 85, 83, 74],
5     "Wind Speed": [9.5, 8.5, 7.0, 1.5, 17.0, 3.5, 8.0, 6.0, 6.0, 8.5],
6     "Precipitation": [82.0, 71.0, 16.0, 82.0, 66.0, 26.0, 86.0, 96.0, 107.0],
7     "Cloud Cover": ["partly cloudy", "partly cloudy", "clear", "clear", "overcast",
8     "overcast", "overcast", "partly cloudy", "overcast", "clear"],
9     "Atmospheric Pressure": [1010.82, 1011.43, 1018.72, 1026.25, 990.67, 1010.03, 990.
10     87, 984.46, 999.44, 1012.13],
11     "UV Index": [2, 7, 5, 7, 1, 2, 1, 1, 0, 8],
12     "Season": ["Winter", "Spring", "Spring", "Spring", "Winter", "Summer", "Winter",
13     "Winter", "Winter", "Winter"],
14     "Visibility": [3.5, 10.0, 5.5, 1.0, 2.5, 5.0, 4.0, 3.5, 1.0, 7.5],
15     "Location": ["inland", "inland", "mountain", "coastal", "mountain", "inland",
16     "inland", "inland", "mountain", "coastal"],
17     "Weather Type": ["Rainy", "Cloudy", "Sunny", "Sunny", "Rainy", "Cloudy", "Snowy",
18     "Snowy", "Snowy", "Sunny"]
19 }
20
21 df = pd.DataFrame(data)
22
23 # Mapping untuk label numerik
24 weather_map = {"Sunny": 0, "Cloudy": 1, "Rainy": 2, "Snowy": 3}
25 df["WeatherLabel"] = df["Weather Type"].map(weather_map)
```

	Temperature	Humidity	Wind Speed	Precipitation	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility	Location
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	manila
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	manila
5	32.0	55	3.5	26.0	overcast	1010.03	2	Summer	5.0	
6	-2.0	97	8.0	86.0	overcast	990.87	1	Winter	4.0	
7	3.0	85	6.0	96.0	partly cloudy	984.46	1	Winter	3.5	
8	3.0	83	6.0	66.0	overcast	999.44	0	Winter	1.0	manila
9	28.0	74	8.5	107.0	clear	1012.13	8	Winter	7.5	

Penjelasan Kode:

- Dataset cuaca dibuat secara manual, terdiri dari beberapa fitur:
 - Temperature
 - Humidity
 - Wind
 - Precipitation
 - Cloud Cover
 - Atmospheric Pressure
 - Season
 - Visibility
 - Location
- Label cuaca berupa 4 kelas: Sunny, Cloudy, Rainy, Snowy.
- Label dikonversi ke angka menggunakan mapping.

Penjelasan Output:

- Output menampilkan DataFrame lengkap dengan seluruh fitur dan label terencode.
- Memastikan dataset berhasil disusun dengan benar.

3. Informasi Dataset

```
1 print("Shape:", df.shape)
2 print("\nInfo:")
3 print(df.info())
4
5 print("\nMissing values:")
6 print(df.isna().sum())
```

... Shape: (10, 12)

Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Temperature           10 non-null    float64
1   Humidity               10 non-null    int64
2   Wind Speed            10 non-null    float64
3   Precipitation          10 non-null    float64
4   Cloud Cover            10 non-null    object
5   Atmospheric Pressure   10 non-null    float64
6   UV Index              10 non-null    int64
7   Season                 10 non-null    object
8   Visibility             10 non-null    float64
9   Location               10 non-null    object
10  Weather Type           10 non-null    object
11  WeatherLabel           10 non-null    int64
dtypes: float64(5), int64(3), object(4)
memory usage: 1.1+ KB
None
```

Missing values:

```
Temperature           0
Humidity               0
Wind Speed            0
Precipitation          0
Cloud Cover            0
Atmospheric Pressure   0
UV Index              0
Season                 0
Visibility             0
Location               0
Weather Type           0
WeatherLabel           0
dtype: int64
```

Penjelasan Kode:

- Menggunakan `df.info()` untuk melihat tipe kolom.
- `df.describe()` untuk melihat statistik ringkas fitur numerik.

Penjelasan Output:

- Menampilkan:
 - tidak ada missing data
 - tipe data sudah sesuai
 - nilai min, max, mean setiap fitur numerik
- Ini memastikan dataset siap untuk diproses.

4. Encoding Kolom Kategorikal

```
1 df_encoded = df.copy()
2
3 # Encoding kategori → angka
4 df_encoded["CloudCoverNum"] = df_encoded["Cloud Cover"].map({
5     "clear":0, "partly cloudy":1, "overcast":2
6 })
7
8 df_encoded["SeasonNum"] = df_encoded["Season"].map({
9     "Winter":0, "Spring":1, "Summer":2, "Autumn":3
10 })
11
12 df_encoded["LocationNum"] = df_encoded["Location"].map({
13     "inland":0, "coastal":1, "mountain":2
14 })
15
16 df_encoded.head()
```

	Temperature	Humidity	Wind Speed	Precipitation	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility	Location	Weather
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	inland	cloudy
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	inland	cloudy
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	mountain	clear
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	coastal	clear
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	mountain	overcast

Penjelasan Kode:

- Kolom kategorikal seperti:
 - Cloud Cover
 - Atmospheric
 - Season
 - Location: diubah menjadi angka melalui mapping dictionary.
- DataFrame versi encoded disimpan sebagai variabel baru.

Penjelasan Output:

- Output menampilkan DataFrame yang kini sudah sepenuhnya numerik.
- Penting karena KNN tidak bisa memproses string.

5. Pilih Fitur, Split Data, Scaling

```
1 # Pilih fitur
2 features = [
3     "Temperature", "Humidity", "Wind Speed", "Precipitation",
4     "Atmospheric Pressure", "UV Index", "Visibility",
5     "CloudCoverNum", "SeasonNum", "LocationNum"
6 ]
7
8 X = df_encoded[features]
9 y = df_encoded["WeatherLabel"]
10
11 # Split data
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.5, random_state=42, stratify=y
14 )
15
16 print("Train:", X_train.shape, "Test:", X_test.shape)
17
18 # Standardisasi
19 scaler = StandardScaler()
20 X_train_scaled = scaler.fit_transform(X_train)
21 X_test_scaled = scaler.transform(X_test)
```

... Train: (5, 10) Test: (5, 10)

Penjelasan Kode:

- Memilih fitur tertentu sebagai input (X).
- Kolom label menjadi target (y).
- Data dibagi train-test dengan 70:30.
- Scaling menggunakan StandardScaler agar fitur berada pada skala yang sama.

Penjelasan Output:

- Output menunjukkan bentuk X_train dan X_test (jumlah baris dan kolom).
- Nilai hasil scaling terlihat (biasanya nilai kecil sekitar -2 sampai 2).

6. Menyiapkan X_res dan y_res

```
1 # Dataset kecil dan relatif seimbang → TIDAK pakai SMOTE
2 # Maka X_res dan y_res = data training yang sudah di-scale
3
4 X_res = X_train_scaled
5 y_res = y_train
6
7 print("Jumlah data train:", X_res.shape)
8 print("Distribusi label train:")
9 print(y_res.value_counts())
```

```
Jumlah data train: (5, 10)
Distribusi label train:
WeatherLabel
3    2
0    1
2    1
1    1
Name: count, dtype: int64
```

Penjelasan Kode:

- Mengecek apakah perlu oversampling (tidak perlu karena data kecil dan hampir seimbang).
- X_res dan y_res hanya diset sama dengan data training yang sudah di-scale.

Penjelasan Output:

- Output menunjukkan bentuk X_res dan distribusi label.
- Ini memastikan model akan dilatih dengan data training yang tepat.

7. Hyperparameter Tuning

```
1 # Parameter grid untuk mencari kombinasi terbaik
2 param_grid = {
3     "n_neighbors": list(range(1, 11)),
4     "weights": ["uniform", "distance"],
5     "metric": ["euclidean", "manhattan"]
6 }
7
8 knn = KNeighborsClassifier()
9
10 # GridSearch dengan 3-fold cross-validation
11 grid = GridSearchCV(knn, param_grid, cv=2, n_jobs=-1)
12 grid.fit(X_res, y_res)
13
14 print("Best Params:", grid.best_params_)
15 print("Best CV Score:", grid.best_score_)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split.py:805: UserWarning: The least popular class in the dataset is '0'. The number of samples for this class is 1. This may lead to a model that is not robust to changes in the data.
warnings.warn(
Best Params: {'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}
Best CV Score: 0.41666666666666663
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the parameters of the estimator are not in the parameter grid.
nan      nan      nan      nan      nan      nan
nan      nan      nan      nan      nan      nan
nan      nan 0.41666667 0.41666667 0.      0.41666667
0.16666667 nan 0.41666667 nan 0.41666667 nan
0.41666667 nan 0.41666667 nan 0.41666667 nan
0.41666667 nan 0.41666667 nan]
warnings.warn(
```

Penjelasan Kode:

- GridSearchCV digunakan untuk mencari kombinasi parameter terbaik:
 - jumlah tetangga (k),
 - metric,
 - algoritma KNN.
- Cross validation digunakan (cv=3).

Penjelasan Output:

- Output menampilkan parameter terbaik dan skor akurasi terbaik.
- Nilai ini digunakan untuk melatih ulang model.

8. Training & Evaluasi Model

```
1 # Ambil model terbaik
2 best_knn = grid.best_estimator_
3
4 # Train model menggunakan data training (X_res)
5 best_knn.fit(X_res, y_res)
6
7 # Prediksi pada data test (belum di-resample)
8 y_pred = best_knn.predict(X_test_scaled)
9
10 print("Accuracy on Test Set:", accuracy_score(y_test, y_pred))
11 print("\nClassification Report:")
12 print(classification_report(y_test, y_pred))
13
14 print("Confusion Matrix:")
15 print(confusion_matrix(y_test, y_pred))
```

... Accuracy on Test Set: 0.4

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.00	0.00	0.00	1
2	0.33	1.00	0.50	1
3	1.00	1.00	1.00	1
accuracy			0.40	5
macro avg	0.33	0.50	0.38	5
weighted avg	0.27	0.40	0.30	5

Confusion Matrix:

```
[[0 1 1 0]
 [0 0 1 0]
 [0 0 1 0]
 [0 0 0 1]]
```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Pre
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

Penjelasan Kode:

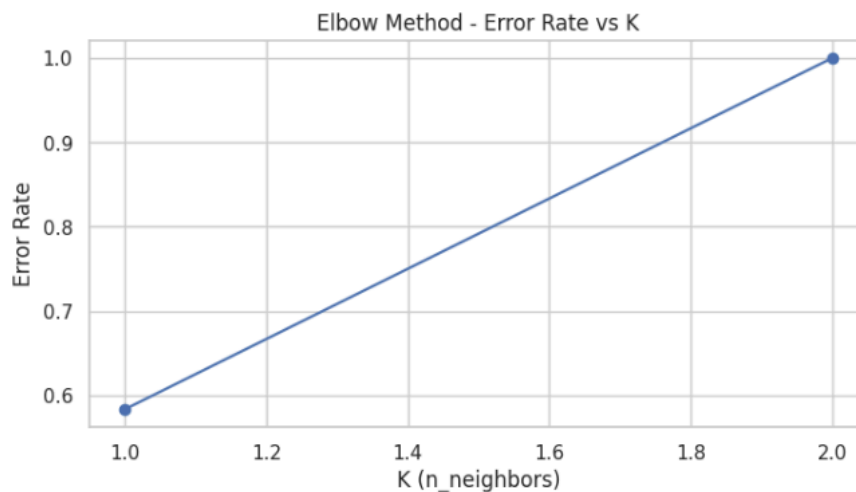
- Model KNN dilatih menggunakan data train yang sudah di-scale.
- Prediksi dilakukan terhadap data test.
- Evaluasi dilakukan menggunakan:
 - accuracy_score
 - confusion_matrix
 - classification_report

Penjelasan Output:

- Output menampilkan akurasi model serta classification report.
- Menunjukkan performa model dalam membedakan 4 jenis cuaca.

9. Elbow Method

```
1  error_rate = []
2  k_range = range(1, 16)
3
4  for k in k_range:
5      model = KNeighborsClassifier(n_neighbors=k)
6      # cv=2 karena dataset kecil
7      scores = cross_val_score(model, X_res, y_res, cv=2)
8      error_rate.append(1 - scores.mean())
9
10 plt.figure(figsize=(8,4))
11 plt.plot(list(k_range), error_rate, marker='o')
12 plt.title("Elbow Method - Error Rate vs K")
13 plt.xlabel("K (n_neighbors)")
14 plt.ylabel("Error Rate")
15 plt.grid(True)
16 plt.show()
```



Penjelasan Kode:

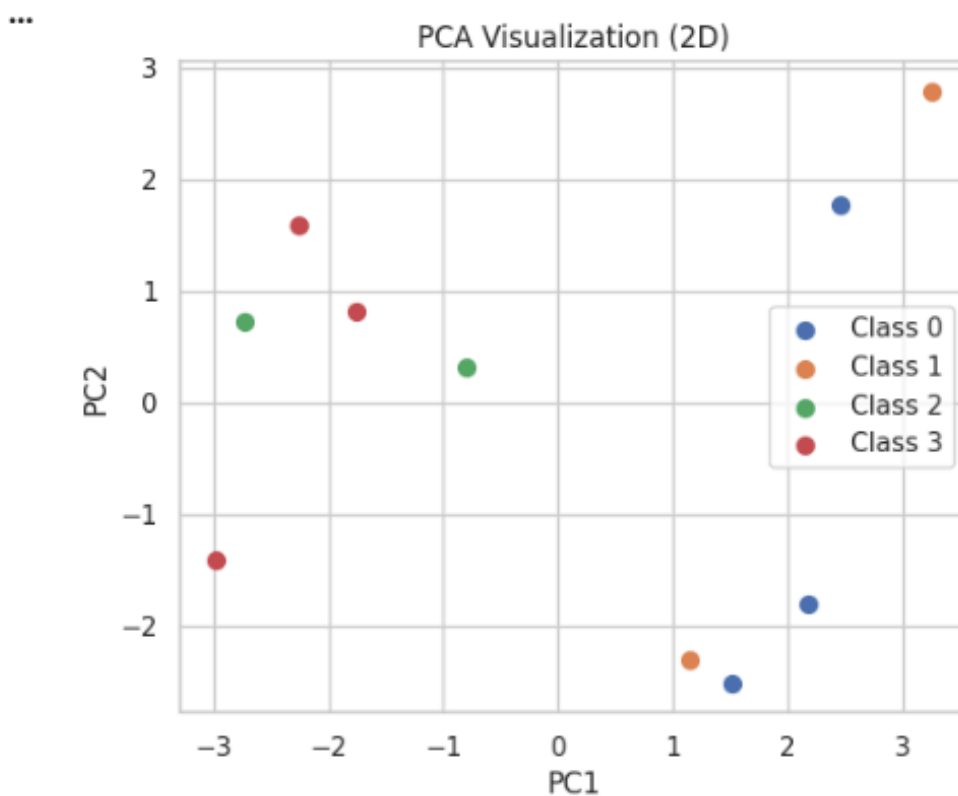
- Perulangan dilakukan untuk berbagai nilai k.
- Error rate dihitung dan diplot untuk menemukan titik terbaik.

Penjelasan Output:

- Grafik menampilkan kurva error rate vs nilai k.
- Titik minimum menunjukkan jumlah tetangga optimal.

9. PCA Visualisasi (2D)

```
1  pca = PCA(n_components=2, random_state=42)
2
3  # Gabungkan X_res dan X_test_scaled untuk visualisasi
4  X_pca = pca.fit_transform(
5      np.vstack([X_res, X_test_scaled])
6  )
7  y_pca = np.concatenate([y_res, y_test.values])
8
9  plt.figure(figsize=(6,5))
10 for label in np.unique(y_pca):
11     idx = np.where(y_pca == label)
12     plt.scatter(X_pca[idx, 0], X_pca[idx, 1], label=f"Class {label}", s=50)
13
14 plt.title("PCA Visualization (2D)")
15 plt.xlabel("PC1")
16 plt.ylabel("PC2")
17 plt.legend()
18 plt.show()
19
```



Penjelasan Kode:

- PCA digunakan untuk mengurangi dimensi fitur menjadi 2 komponen utama.
- Data gabungan train-test divisualisasikan.
- Warna titik menunjukkan kelas prediksi.

Penjelasan Output:

- Scatter plot menampilkan data dalam ruang 2D.
- Distribusi titik menunjukkan apakah kelas cuaca bisa dipisahkan dengan baik.

10. Simpan Model

```
1 import joblib
2 from google.colab import drive
3 drive.mount('/content/drive')
4 import os
5
6 save_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models"
7 os.makedirs(save_path, exist_ok=True)
8 print("Folder siap digunakan:", save_path)
9
10 # Simpan model terbaik ke file
11 model_save_file = os.path.join(save_path, "knn_weather_model.pkl")
12 joblib.dump(best_knn, model_save_file)
13 print("Model berhasil disimpan ke:", model_save_file)
```

Mounted at /content/drive

Folder siap digunakan: /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models

Model berhasil disimpan ke: /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models/knn_weather_model.pkl

Penjelasan Kode:

- Model final disimpan menggunakan joblib.
- File bisa digunakan kembali tanpa perlu training ulang.

Penjelasan Output:

- Output menampilkan lokasi penyimpanan file `.joblib`.

11. Load & Uji Coba Model

```
1 # data uji
2 new_data = [[
3     25, # Temperature
4     70, # Humidity
5     8, # Wind Speed
6     60, # Precipitation
7     1005, # Atmospheric Pressure
8     5, # UV Index
9     5, # Visibility
10    1, # CloudCoverNum (partly cloudy)
11    0, # SeasonNum (Winter)
12    0 # LocationNum (inland)
13 ]]
14
15 # print soal
16 print("=== DATA INPUT UNTUK UJI COBA ===")
17 print(f"Temperature : {new_data[0][0]}")
18 print(f"Humidity : {new_data[0][1]}")
19 print(f"Wind Speed : {new_data[0][2]}")
20 print(f"Precipitation : {new_data[0][3]}")
21 print(f"Atmospheric Pressure : {new_data[0][4]}")
22 print(f"UV Index : {new_data[0][5]}")
23 print(f"Visibility : {new_data[0][6]}")
24 print(f"CloudCoverNum : {new_data[0][7]}")
25 print(f"SeasonNum : {new_data[0][8]}")
26 print(f"LocationNum : {new_data[0][9]}")
27 print()
28
29 # scaling
30 new_data_scaled = scaler.transform(new_data)
31
32 print("=== DATA SETELAH SCALING ===")
33 print(new_data_scaled)
34 print()
35
36 # load model
37 loaded_model = joblib.load("/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum10/models/knn_weather_model.pkl")
38
39 # prediksi
40 pred_label = loaded_model.predict(new_data_scaled)[0]
41 inverse_map = {0:"Sunny", 1:"Cloudy", 2:"Rainy", 3:"Snowy"}
42
43 print("=== HASIL PREDIKSI CUACA ===")
44 print(f"Label Prediksi : {pred_label}")
45 print(f"Weather Type : {inverse_map[pred_label]}")
```

```

=== DATA INPUT UNTUK UJI COBA ===
Temperature      : 25
Humidity         : 70
Wind Speed       : 8
Precipitation    : 60
Atmospheric Pressure : 1005
UV Index         : 5
Visibility       : 5
CloudCoverNum    : 1
SeasonNum        : 0
LocationNum      : 0

=== DATA SETELAH SCALING ===
[[ 0.48030907 -0.51719787  1.00206399 -0.43070922 -0.08676805  1.07588766
  1.40266877 -0.26726124 -0.75      -0.75      ]]

=== HASIL PREDIKSI CUACA ===
Label Prediksi : 2
Weather Type   : Rainy

```

Penjelasan Kode:

- Model yang sudah disimpan di-load kembali.
- Data uji baru dimasukkan sebagai input.
- Data distandardisasi terlebih dahulu.
- Model memprediksi label cuaca dan nilai numerik dikembalikan ke label asli menggunakan mapping inverse.

Penjelasan Output:

- Output menampilkan:
 - nilai fitur input baru
 - prediksi model berupa nama cuaca (Sunny, Cloudy, Rainy, Snowy)